In [ ]:
```
import marimo as mo
```

Conway's Game of Life — tabuleiro finito

Espaço de estados é definido por uma grelha de células booleanas de dimensão N * N (com N > 16), estas N^2 são referidas como "normais" Inicialmente todas as células normais estão mortas menos as células 1 <= i,j <= 3, que estão vivas. Dá-se o estado de erro quando todas as células normais estão mortas.

Uma vez que a grelha devia de ser infinita, implementamos umas regras para que seja finito:

- (células iniciais) 2N + 1 corresponde a um dos indices, i ou j tem de ser 0. Estas células tem valores constantes, que vão ser gerados aleatoriamente com um probabilidade 1/2 de estarem vivas.
- (células finais) 2N + 3 corresponde a um dos indices, i ou j tem que ter o valor de N + 1. As células finais tem o valor constante de zero (estão mortas)

Regra "B3|S23"

- Nasce se tiver EXATAMENTE 3 vizinhos vivos
- Sobrevie se tiver 2 ou 3 vizinhos vivos Caso contrário morre

As células (i0, j0), (i1, j1) são vizinhas se e só se max(|i0 - i1|, |j0 - j1|) = 1

In [ ]:
```
# ALTERADO: o marimo não aceita "from pysmt.shortcuts import *",
# por isso os nomes usados são importados um a um.
import random
from pysmt.shortcuts import (
    BOOL, Bool, Int, TRUE, Symbol,
    And, Or, Not, Iff, Ite, Plus, Equals, ExactlyOne,
    Solver,
)
```

In [ ]:
```
#Sabemos que temos que ter um N > 16
N = 17
#Podemos mudar para gerar outra grelha inicial
seed = 3
# Parâmetro que escolhe a relação de transição:
#   "sincrono"   -> todas as células normais transitam em simultâneo
#   "assincrono" -> uma e só uma célula normal transita
MODO = "sincrono"
```

In [ ]:
```
def declare(inst, N):
    estado = {}
    estado['grelha'] = {
        (i, j): Symbol(f"C_{inst}_{i}_{j}", BOOL)
```

```
        for i in range(N + 2) for j in range(N + 2)
    }
    return estado

def init(estado, N):
    rng = random.Random(seed)
    restricoes = []
    for (i,j), c in estado['grelha'].items():
        if i == 0 or j == 0:
            valor_c = rng.randint(0,1) == 1
        elif i == N+1 or j == N+1:
            valor_c = False
        else:
            valor_c = True if i <= 3 and j <= 3 else False
        restricoes.append(Iff(c, Bool(valor_c)))
    return And(restricoes)


def conta_vizinhos(atual, i, j):
    vizinhos = []
    for x in range(i-1, i+2):
        for y in range(j-1, j+2):
            if (x,y) != (i,j):
                vizinhos.append(Ite(atual['grelha'][(x,y)], Int(1),
Int(0)))
    return Plus(vizinhos)

# ALTERADO: recebe t (o número da transição), para as duas transições
# terem a mesma assinatura. Aqui o t não é usado.
def transicoes(atual, proximo, N, t):
    restricoes = []
    #bordas
    for ij in range(N+2):
        restricoes.append(Iff(atual['grelha'][(ij,0)],proximo['grelha']
[(ij,0)]))
        restricoes.append(Iff(atual['grelha'][(0,ij)],proximo['grelha']
[(0,ij)]))
        restricoes.append(Iff(atual['grelha'][(ij,N+1)],proximo['grelha']
[(ij,N+1)]))
        restricoes.append(Iff(atual['grelha'][(N+1,ij)],proximo['grelha']
[(N+1,ij)]))

    for i in range(1, N+1):
        for j in range(1, N+1):
            vizinhos_vivos = conta_vizinhos(atual, i, j)
            restricoes.append(Iff(proximo['grelha'][(i,j)],
Or(Equals(vizinhos_vivos, Int(3)), And(atual['grelha'][(i,j)],
Equals(vizinhos_vivos, Int(2))))))
    return And(restricoes)

# ALTERADO: recebe t. Antes o t era usado no nome das escolhas mas não
existia (NameError).
def transicoes_Uma(atual, proximo, N, t):
    restricoes = []
    #bordas
    for ij in range(N+2):
```

```
        restricoes.append(Iff(atual['grelha'][(ij,0)],proximo['grelha']
[(ij,0)]))
        restricoes.append(Iff(atual['grelha'][(0,ij)],proximo['grelha']
[(0,ij)]))
        restricoes.append(Iff(atual['grelha'][(ij,N+1)],proximo['grelha']
[(ij,N+1)]))
        restricoes.append(Iff(atual['grelha'][(N+1,ij)],proximo['grelha']
[(N+1,ij)]))

    escolha = {(i, j): Symbol(f"esc_{t}_{i}_{j}", BOOL)
               for i in range(1, N+1) for j in range(1, N+1)}
    restricoes.append(ExactlyOne(escolha.values()))


    for i in range(1, N+1):
        for j in range(1, N+1):
            vizinhos_vivos = conta_vizinhos(atual, i, j)
            restricoes.append(
                Ite(escolha[(i,j)],
                    Iff(proximo['grelha'][(i,j)],
                        Or(Equals(vizinhos_vivos, Int(3)),
And(atual['grelha'][(i,j)], Equals(vizinhos_vivos, Int(2))))
                       ),
                    Iff(proximo['grelha'][(i,j)], atual['grelha'][(i,j)])
                   )
            )
    return And(restricoes)
```

In [ ]:

```
def modelo(declare, init, trans, k, N, prop):
    with Solver(name="z3") as s:
        estados = [declare(inst, N) for inst in range(k + 1)]

        restricoes = []
        formula_inicial = init(estados[0], N)
        restricoes.append(formula_inicial)

        # Para todos os outros tem que ser uma transição valida para todos
os outros
        for i in range(k):
            # ALTERADO: passa também o número da transição (i)
            restricoes.append(trans(estados[i], estados[i + 1], N, i))


        restricoes.append(Or(prop(estados, N)))

        s.add_assertion(And(restricoes))


        #Encontramos um estado de erro
        if s.solve():
            model = s.get_model()
            return estados, model
        else:
            # ALTERADO: mensagem genérica, porque o modelo também serve
para o 2.2
            print(f"Não foi encontrado nenhum contraexemplo com {k}
```

```
iterações")
            return estados, None
```

In [ ]:
```
# Estado de erro, todas as células normais estarem mortas
def erro(estado, N):
    restricoes = []
    for i in range(1, N+1):
        for j in range(1, N+1):
            restricoes.append(Iff(estado['grelha'][(i,j)], Bool(False)))
    return And(restricoes)

def alcanca_erro(estados, N):
    return Or([erro(estado, N) for estado in estados])

# ALTERADO: retirado o parâmetro k, que não era usado
# (e dependia de um k global, que no marimo não existe).
def verificar_estados(estados, vivo, i, j):
    celulas = [estado['grelha'][(i,j)] for estado in estados]
    return And(celulas) if vivo else And([Not(celula) for celula in
celulas])

def muda_de_estado(estados, N):
    celulas = []
    for i in range(1, N+1):
        for j in range(1, N+1):
            celulas.append(Or(verificar_estados(estados, True, i, j),
verificar_estados(estados, False, i, j)))

    return Or(celulas)

def estados_iguais(e1, e2, N):
    iguais = []
    for i in range(1, N+1):
        for j in range(1, N+1):
            iguais.append(Iff(e1['grelha'][(i,j)], e2['grelha'][(i,j)]))
    return And(iguais)

def fecha_laco(estados, N):
    k = len(estados) - 1
    return Or([estados_iguais(estados[k], estados[l], N) for l in
range(k)])

def celula_permanente(estados, N):
    return And(fecha_laco(estados, N), muda_de_estado(estados, N))
```

In [ ]:
```
# A relação de transição usada em todo o notebook é escolhida pelo
parâmetro MODO.
TRANSICOES = {"sincrono": transicoes, "assincrono": transicoes_Uma}
trans = TRANSICOES[MODO]
```

Apresentação de tabuleiros e contraexemplos

As funções seguintes só **leem** o modelo devolvido pelo solver e desenham-no. Já não há incógnitas nesta fase: o solver já escolheu todos os valores, por isso podemos usar `if` e `==` normais do Python.

In [ ]:

```
import time

# Cores do desenho (o tabuleiro tem fundo próprio, por isso lê-se bem no
tema claro e no escuro)
COR_VIVA = "#111827"         # célula normal viva
COR_MORTA = "#ffffff"        # célula normal morta
COR_BORDA_VIVA = "#6b7280"   # célula inicial/final viva
COR_BORDA_MORTA = "#e5e7eb"  # célula inicial/final morta
COR_DESTAQUE = "#dc2626"     # célula para onde o contraexemplo aponta


def sem_restricao(estados, N):
    """'Propriedade' sempre verdadeira: serve para pedir ao modelo um traço
qualquer."""
    return TRUE()


# ---------- ler o modelo ----------
def ler_modelo(model):
    """Copia os valores do modelo para um dicionário (símbolo ->
True/False).
    No pySMT, cada leitura do modelo percorre-o todo; com milhares de
variáveis
    (modo assíncrono) isso fica muito lento. Copiar uma vez resolve."""
    return {s: v.is_true() for s, v in model if v.is_bool_constant()}

def valor(valores, var):
    return valores.get(var, False)

def fotografia(valores, estado, N):
    """As células normais de um estado como um tuplo de booleanos (para
comparar estados)."""
    return tuple(valor(valores, estado['grelha'][(i, j)])
                 for i in range(1, N + 1) for j in range(1, N + 1))

def analisar_laco(fotos):
    """Se o último estado repetir um anterior, devolve (l, p, entrada):
    l = estado anterior mais próximo igual ao último, p = comprimento real
do ciclo,
    entrada = primeiro estado que já está dentro do ciclo."""
    k = len(fotos) - 1
    for l in range(k - 1, -1, -1):
        if fotos[l] == fotos[k]:
            p = k - l
            entrada = next(t for t in range(k) if fotos[t] == fotos[t + p])
            return l, p, entrada
    return None

def celulas_constantes(valores, estados, N):
    """Células normais com o mesmo valor em todos os estados do traço
(interpretação I1)."""
```

```
    sempre_vivas, sempre_mortas = [], []
    for i in range(1, N + 1):
        for j in range(1, N + 1):
            vistos = {valor(valores, e['grelha'][(i, j)]) for e in estados}
            if vistos == {True}:
                sempre_vivas.append((i, j))
            elif vistos == {False}:
                sempre_mortas.append((i, j))
    return sempre_vivas, sempre_mortas

def celula_escolhida(valores, t, N):
    """Modo assíncrono: célula escolhida na transição t.
    No pySMT o mesmo nome dá o mesmo símbolo, por isso basta recriar o nome
usado em transicoes_Uma."""
    for i in range(1, N + 1):
        for j in range(1, N + 1):
            s = Symbol(f"esc_{t}_{i}_{j}", BOOL)
            if valor(valores, s):
                return (i, j)
    return None


# ---------- desenhar ----------
def html_estado(valores, estado, N, titulo="", destaque=None, tamanho=11):
    quadrados = []
    for i in range(N + 2):
        for j in range(N + 2):
            viva = valor(valores, estado['grelha'][(i, j)])
            borda = i in (0, N + 1) or j in (0, N + 1)
            if borda:
                cor = COR_BORDA_VIVA if viva else COR_BORDA_MORTA
            else:
                cor = COR_VIVA if viva else COR_MORTA
            extra = f"box-shadow: inset 0 0 0 2px {COR_DESTAQUE};" if (i,
j) == destaque else ""
            quadrados.append(f'<div title="({i},{j})" style="background:
{cor};{extra}"></div>')
    return (
        '<div style="background:#fff;color:#111827;padding:8px;border:1px
solid #d1d5db;'
        'border-radius:6px;font-family:sans-serif">'
        f'<div style="font-size:12px;font-weight:600;margin-bottom:6px">
{titulo}</div>'
        f'<div style="display:grid;grid-template-columns:repeat({N + 2},
{tamanho}px);'
        f'grid-auto-rows:{tamanho}px;gap:1px;background:#d1d5db;border:1px
solid #d1d5db">'
        + "".join(quadrados) + "</div></div>"
    )

def legenda():
    def item(cor, texto, extra=""):
        return (f'<span style="display:inline-flex;align-
items:center;gap:4px;margin-right:14px">'
                f'<span style="width:12px;height:12px;background:
{cor};border:1px solid #9ca3af;{extra}"></span>{texto}</span>')
```

```
    return mo.Html(
        '<div style="font-size:13px;margin:4px 0 8px 0">'
        + item(COR_VIVA, "viva") + item(COR_MORTA, "morta")
        + item(COR_BORDA_VIVA, "borda viva") + item(COR_BORDA_MORTA, "borda
morta")
        + item(COR_MORTA, "célula destacada", f"box-shadow: inset 0 0 0 2px
{COR_DESTAQUE}")
        + "</div>"
    )

def html_traco(model, estados, N, assincrono=False, laco=False,
destaque=None, maximo=12):
    """Desenha o traço. Em traços longos mostra só o início, a entrada no
ciclo e o fim."""
    valores = model if isinstance(model, dict) else ler_modelo(model)
    fotos = [fotografia(valores, e, N) for e in estados]
    info = analisar_laco(fotos) if laco else None
    l, entrada = (info[0], info[2]) if info else (None, None)
    k = len(estados) - 1

    indices = list(range(len(estados)))
    if len(indices) > maximo:
        mostrar = set(range(3)) | set(range(len(estados) - (maximo - 3),
len(estados)))
        if entrada is not None:
            mostrar.add(entrada)
        indices = sorted(mostrar)

    blocos = []
    for t in indices:
        titulo = f"Estado {t}"
        if assincrono and t > 0:
            titulo += f" · escolhida {celula_escolhida(valores, t - 1, N)}"
        if not any(fotos[t]):
            titulo += " · ERRO"
        if entrada is not None and t == entrada:
            titulo += " · entra no ciclo"
        if l is not None and t == k:
            titulo += f" · = estado {l}"
        blocos.append(html_estado(valores, estados[t], N, titulo,
destaque))

    nota = ""
    if len(indices) < len(estados):
        nota = (f'<div style="font-size:12px;margin-bottom:6px">Traço
longo: '
                f'{len(estados) - len(indices)} estados omitidos.</div>')
    html = nota + '<div style="display:flex;flex-wrap:wrap;gap:10px">' +
"".join(blocos) + "</div>"
    return mo.Html(html), info


# ---------- procurar e apresentar ----------
def plural(n, singular, plural_):
    return f"{n} {singular if n == 1 else plural_}"
```

```python
def procurar(trans, ks, prop, N):
    """Chama o teu modelo para cada k, por ordem, e pára no primeiro que dá contraexemplo."""
    registo = []
    for k in ks:
        inicio = time.time()
        estados, model = modelo(declare, init, trans, k, N, prop)
        registo.append(f"k = {k}: {'contraexemplo' if model is not None else 'nenhum'} "
                       f"({time.time() - inicio:.1f} s)")
        if model is not None:
            return k, estados, model, registo
    # sem contraexemplo: devolve o maior k experimentado, para a mensagem
    return ks[-1], None, None, registo

def apresentar_2_1(resultado, N, assincrono):
    k, estados, model, registo = resultado
    texto = "**Procura:** " + " · ".join(registo)
    if model is None:
        return mo.md(texto + f"\n\nNenhum traço até k = {k} alcança o estado de erro. "
                     "Isto não prova o *nunca*: só diz que não acontece nesses passos.")
    desenho, _ = html_traco(model, estados, N, assincrono=assincrono)
    return mo.vstack([
        mo.md(texto + f"\n\n**Contraexemplo** com {plural(k, 'transição', 'transições')}: "
              "o último estado é de erro."),
        legenda(), desenho,
    ])

def apresentar_2_2(resultado, N, assincrono):
    k, estados, model, registo = resultado
    texto = "**Procura:** " + " · ".join(registo)
    if model is None:
        return mo.md(texto + "\n\nNenhum laço com estes valores de k serve de contraexemplo.")
    valores = ler_modelo(model)
    vivas, mortas = celulas_constantes(valores, estados, N)
    destaque = vivas[0] if vivas else mortas[0]   # as sempre vivas são mais raras: têm prioridade
    desenho, (l, p, entrada) = html_traco(valores, estados, N, assincrono=assincrono,
                                          laco=True, destaque=destaque)
    linhas = [
        texto + "\n",
        f"**Contraexemplo** com {plural(k, 'transição', 'transições')} (interpretação I1).\n",
        f"- Células sempre vivas: {len(vivas)} · sempre mortas: {len(mortas)}",
        f"- Célula destacada: {destaque} ({'sempre viva' if destaque in vivas else 'sempre morta'})",
        f"- O estado {k} é igual ao estado {l}: a execução entra no ciclo no estado {entrada} "
        f"e repete um ciclo de {plural(p, 'estado', 'estados')} para sempre.",
```

```
    ]
    if assincrono:
        sem_efeito = [t for t in range(k)
                      if fotografia(valores, estados[t], N) ==
fotografia(valores, estados[t + 1], N)]
        if sem_efeito:
            linhas.append(f"- Transições em que a célula escolhida não
mudou nada: {sem_efeito}. "
                          "É a questão da justiça (*fairness*): o
escalonador pode ignorar células para sempre.")
    else:
        linhas.append("- No modo síncrono só existe esta execução, por isso
o resultado é "
                      "definitivo para esta semente.")
    return mo.vstack([mo.md("\n".join(linhas)), legenda(), desenho])

def comprimento_do_laco(trans, N, limite=5000):
    """Só para o modo síncrono (uma única execução): avança um passo de
cada vez com a
    tua relação de transição e devolve o k em que o traço fecha o laço.
    A transição é construída uma vez; em cada passo o estado atual é dado
ao solver
    como suposição (assumptions), e o solver devolve o estado seguinte."""
    e0 = declare(0, N)
    with Solver(name="z3") as s:
        s.add_assertion(init(e0, N))
        s.solve()
        valores = {c: s.get_py_value(v) for c, v in e0['grelha'].items()}
    atual, proximo = declare("atual", N), declare("proximo", N)
    vistos = {}
    with Solver(name="z3") as s:
        s.add_assertion(trans(atual, proximo, N, 0))
        for t in range(limite):
            foto = tuple(valores[(i, j)] for i in range(1, N + 1) for j in
range(1, N + 1))
            if foto in vistos:
                return t
            vistos[foto] = t
            s.solve(assumptions=[v if valores[c] else Not(v) for c, v in
atual['grelha'].items()])
            passo = ler_modelo(s.get_model())
            valores = {c: valor(passo, v) for c, v in
proximo['grelha'].items()}
    return None
```

Exemplo de tabuleiro

O estado inicial (com a borda gerada pela `seed`) e as primeiras gerações, na relação de transição escolhida pelo `MODO`.

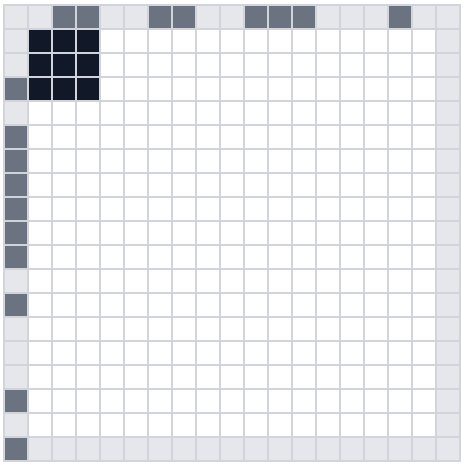
In [ ]:
```
_estados, _model = modelo(declare, init, trans, 11, N, sem_restricao)
_desenho, _ = html_traco(_model, _estados, N, assincrono=(MODO ==
```

```
"assincrono"))
mo.vstack([mo.md(f"Modo **{MODO}**, N = {N}"), legenda(), _desenho])
```

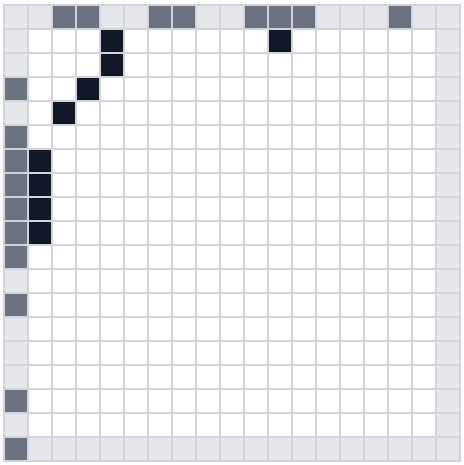
Modo **sincrono**, N = 17

viva morta borda viva borda morta célula destacada

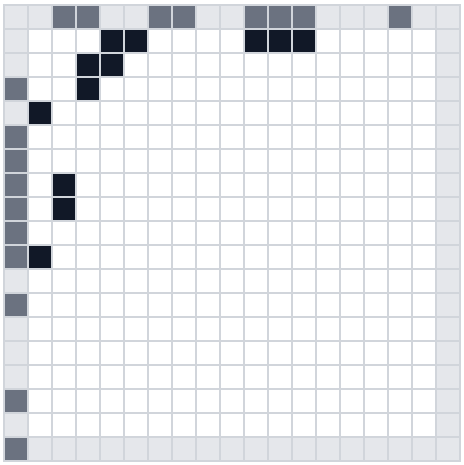
Estado 0



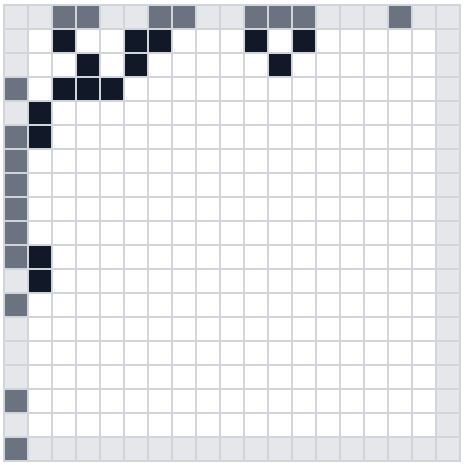
Estado 1



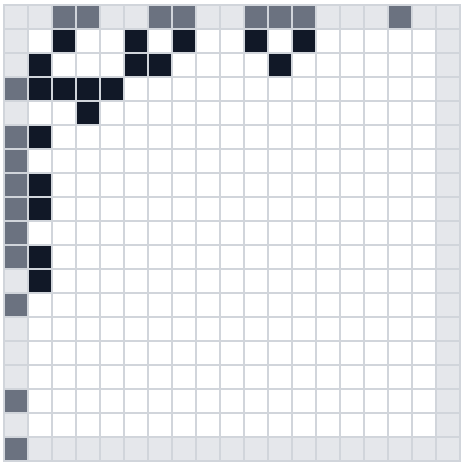
Estado 2



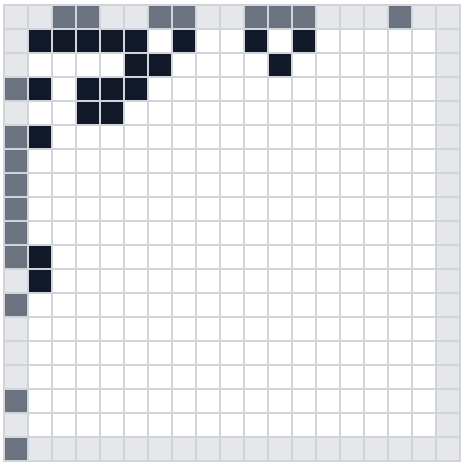
Estado 3



Estado 4

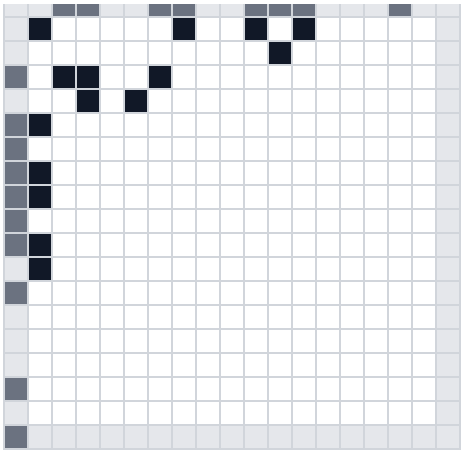


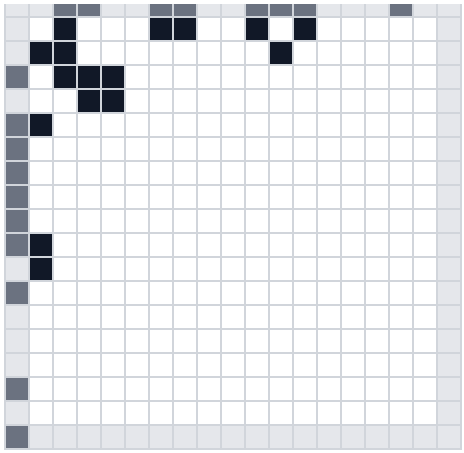
Estado 5



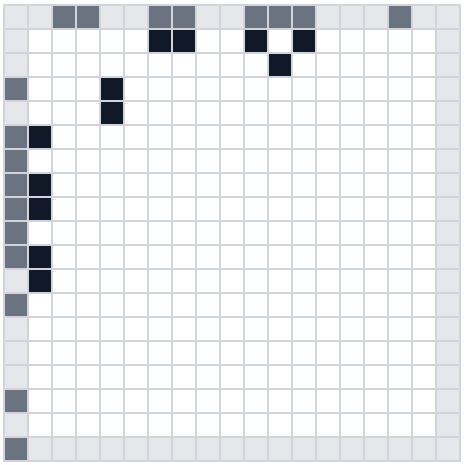
Estado 6

Estado 7

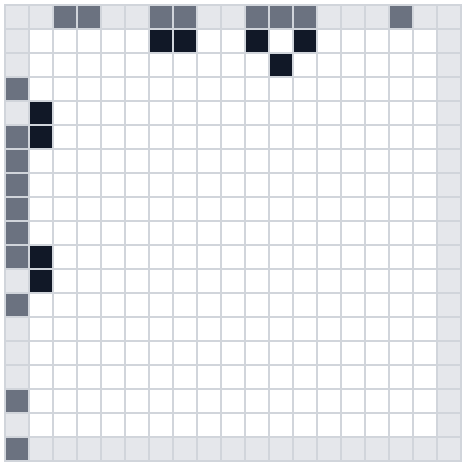




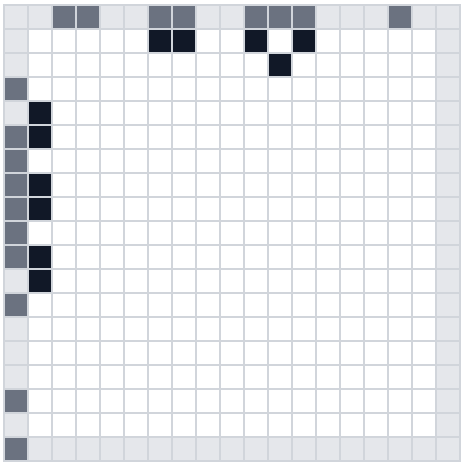
Estado 8



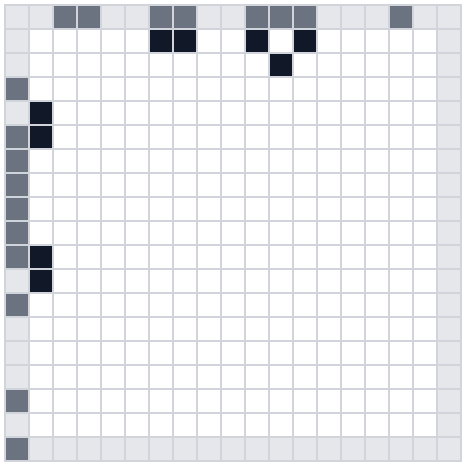
Estado 9



Estado 10



Estado 11



2.1 — Nunca se alcança o estado de erro

Procuramos um traço que **passe** pelo estado de erro. Se existir, é um contraexemplo. No modo assíncrono, o mínimo possível é k = 9: há 9 células vivas e só uma muda por transição.

In [ ]:
```
_ks = [8, 16, 32] if MODO == "sincrono" else [9, 10, 11, 12, 13]
apresentar_2_1(procurar(trans, _ks, alcanca_erro, N), N, assincrono=(MODO
== "assincrono"))
```

```
Não foi encontrado nenhum contraexemplo com 8 iterações
Não foi encontrado nenhum contraexemplo com 16 iterações
Não foi encontrado nenhum contraexemplo com 32 iterações
```

Procura: k = 8: nenhum (0.7 s) · k = 16: nenhum (1.9 s) · k = 32: nenhum (3.9 s) Nenhum traço até k = 32 alcança o estado de erro. Isto não prova o *nunca*: só diz que não acontece nesses passos.

2.2 — Nenhuma célula normal está permanentemente morta ou viva

Interpretação I1: "permanentemente" = em todos os estados da execução. Um contraexemplo é um **laço** (o último estado repete um anterior) em que alguma célula normal tem sempre o mesmo valor.

In [ ]:
```
if MODO == "assincrono":
    _saida = apresentar_2_2(procurar(trans, [1, 2, 4], celula_permanente,
N), N, assincrono=True)
else:
    _saida = mo.md("*(O modo síncrono está na secção seguinte.)*")
_saida
```

(O modo síncrono está na secção seguinte.)

In [ ]:
```
botao_22 = mo.ui.run_button(label="Procurar o laço e verificar o 2.2 (modo
síncrono)")
mo.vstack([
    mo.md(f"""
### 2.2 no modo síncrono

Como só existe uma execução, primeiro descobrimos em que passo ela se
repete
(`comprimento_do_laco`, leve em memória). Depois o teu `modelo` com esse k
e a
`celula_permanente` confirma o contraexemplo.

**Atenção:** o laço pode ser muito longo. Com `seed = 4` só fecha em k =
1247:
a procura e a confirmação demoram uns 2 minutos cada (cerca de 5 minutos no
total) e usam uns 4 GB de memória.
Com `seed = 3` fecha em k = 11 e demora segundos. (Semente atual: {seed}.)
"""),
    botao_22 if MODO == "sincrono" else mo.md("*(Disponível com `MODO =
\"sincrono\"`.)*"),
])
```

Como só existe uma execução, primeiro descobrimos em que passo ela se repete (, leve em memória). Depois o teu
com esse k e a confirma o contraexemplo.

o laço pode ser muito longo. Com só fecha em k = 1247: a procura e a confirmação demoram uns 2 minutos cada (cerca de
5 minutos no total) e usam uns 4 GB de memória. Com fecha em k = 11 e demora segundos. (Semente atual: 3.)

Procurar o laço e verificar o 2.2 (modo
síncrono)

In [ ]:
```
mo.stop(MODO != "sincrono" or not botao_22.value)
_inicio = time.time()
_k = comprimento_do_laco(trans, N)
_procura = mo.md(f"O traço fecha o laço com **k = {_k}** (descoberto em
{time.time() - _inicio:.0f} s).")
mo.vstack([_procura, apresentar_2_2(procurar(trans, [_k],
celula_permanente, N), N, assincrono=False)])
```

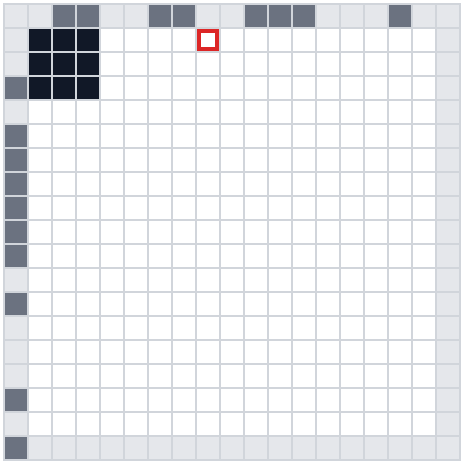
O traço fecha o laço com **k = 11** (descoberto em 4 s).

Procura: k = 11: contraexemplo (1.4 s) **Contraexemplo** com 11 transições (interpretação I1).

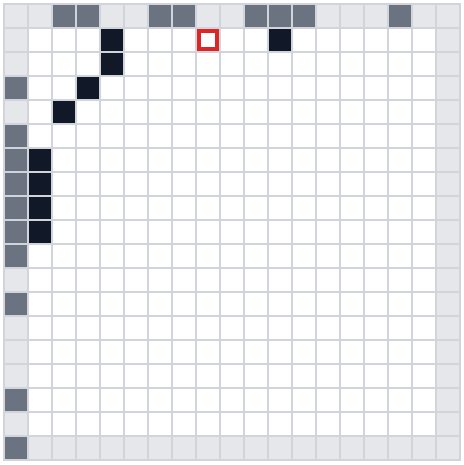
- Células sempre vivas: 0 · sempre mortas: 252
- Célula destacada: (1, 8) (sempre morta)
- O estado 11 é igual ao estado 9: a execução entra no ciclo no estado 9 e repete um ciclo de 2 estados para sempre.
- No modo síncrono só existe esta execução, por isso o resultado é definitivo para esta semente.

viva morta borda viva borda morta célula destacada

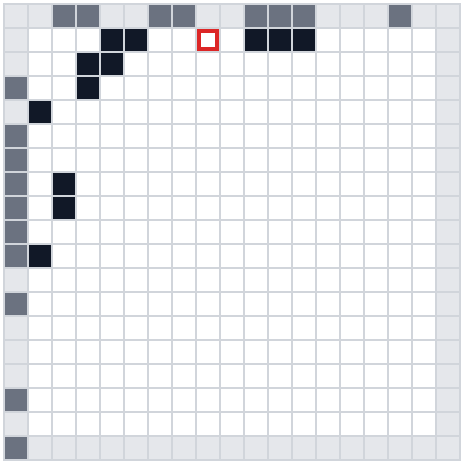
Estado 0



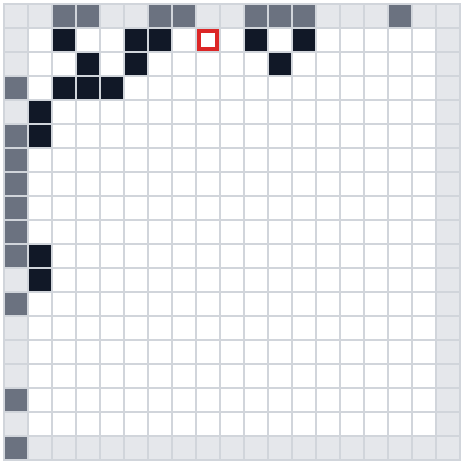
Estado 1



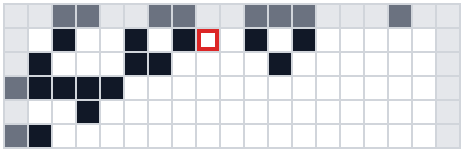
Estado 2



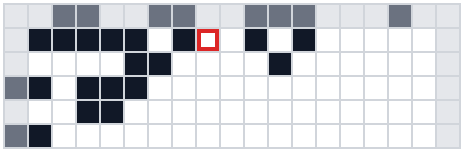
Estado 3



Estado 4



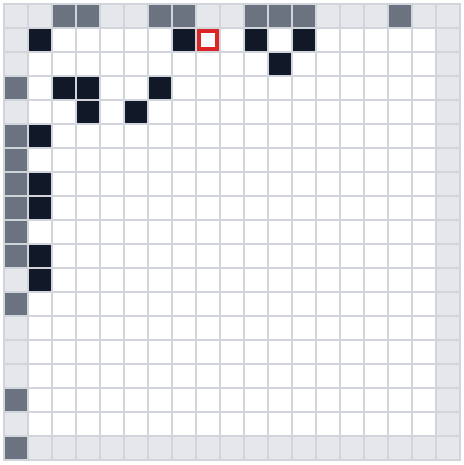
Estado 5



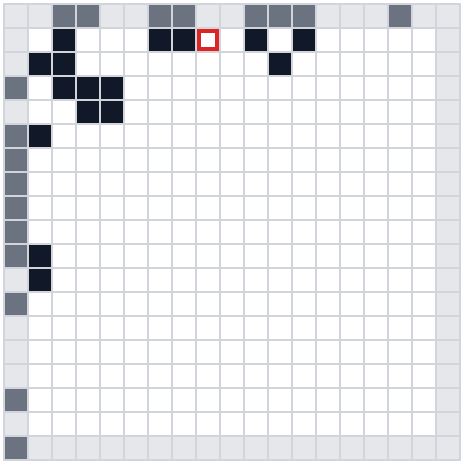




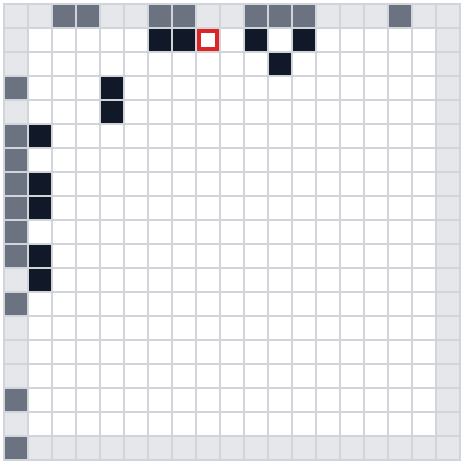
Estado 6



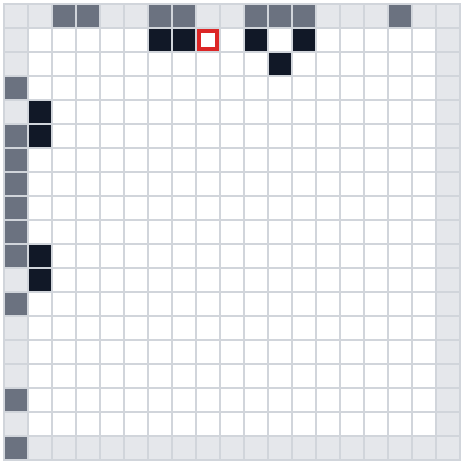
Estado 7



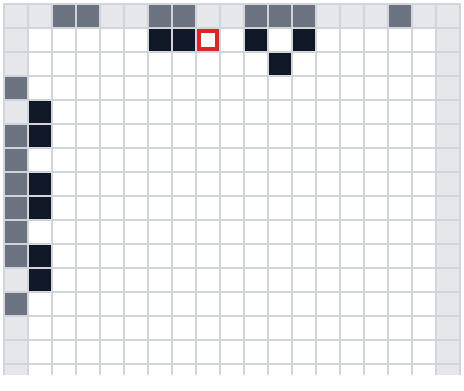
Estado 8



Estado 9 · entra no ciclo



Estado 10



Estado 11 · = estado 9

